



NurvexThink

Software, built and published.

Products Schema (PR 1 of 3) — Implementation Plan

Migration 0003: categories, lifecycle/status split, tech rename, feature/junction tables, security fixes

Date: 2026-07-04
Status: Implementation plan (PR 1 of 3)
Owner: Fatima Abdul Raheem (CEO)
Branch: feat/products

Contents

1	Overview	2
2	Task 1: Migration 0003	3
3	Task 2: TypeScript Layer — Row Types, Public Queries (TDD), Admin Surfaces	8
4	Task 3: Apply Migration, Verify Live, Document, PR	16
5	Self-Review (done)	18

1 Overview

For agentic workers: REQUIRED SUB-SKILL — use superpowers: subagent-driven-development (recommended) or superpowers: executing-plans to implement this plan task-by-task.

Goal: Migration 0003 transforms the live products schema (categories table, lifecycle/status split, tech rename, feature/junction tables, security fixes) while every existing page — public grid, detail, and the basic admin CRUD — keeps working unchanged.

Architecture: One idempotent SQL migration (matching the 0001/0002 style: IF NOT EXISTS + DO \$\$ guards for renames) plus a single TypeScript compile-unit update: row types → public queries → admin surfaces. The UI view-model Product keeps its exact shape (status: "Live"|"Beta"|"Soon" tags) so **no public component changes**; only the mapping layer changes underneath.

Tech Stack: Postgres (Supabase), TypeScript, Next.js 16 Server Actions, Vitest.

Global Constraints

- Brand spelling: **NurvexThink** / nurvexthink (never other spellings).
- RLS default-deny on every table; index every column an RLS policy references (CLAUDE.md).
- service_role key server-side only; never in client code or the repo.
- Conventional Commits; branch feat/products (already checked out; spec commits live on it).
- **No new npm dependencies in this PR.**
- The live DB (project axbsghyqhhdaiylcksbv) has 6 published seed products that must survive: migration is data-preserving, IF NOT EXISTS/DO \$\$-guarded, and safe to re-run.
- Spec: docs/superpowers/specs/2026-07-04-products-experience-design.md (rev 3) — §4 is the contract for this PR.
- Definition of green: npm run typecheck && npm run lint && npm test && npm run build all exit 0.

File Structure

- supabase/migrations/0003_products_experience.sql — **create:** the whole schema transform (§4 of spec).
- src/lib/supabase/types.ts — **modify:** ProductStatusRow → ProductRow (new columns), add ProductCategoryRow, ProductFeatureRow, ProductBlogLinkRow; extend Database.
- src/lib/queries.ts — **modify:** toProduct maps lifecycle→status, tech→tags; filters/order use status/sort_order. Export toProduct for tests.
- src/lib/queries.test.ts — **create:** unit tests for the mapping.
- src/lib/admin-queries.ts — **modify:** type-name update only.
- src/app/admin/(panel)/products/actions.ts — **modify:** form parsing writes lifecycle/tech/status(+
- src/components/admin/product-form.tsx — **modify:** Status select → Lifecycle select; Published checkbox drives status.
- src/app/admin/(panel)/products/page.tsx — **modify:** table reads lifecycle + status.
- CLAUDE.md — **modify:** data-model line mentions the three new tables.

2 Task 1: Migration 0003

Files

- Create: supabase/migrations/0003_products_experience.sql

Interfaces

- Consumes: live schema from 0001_initial_schema.sql + 0002_content_columns_and_seed.sql (products: published bool, status text 'Live', category text, tags text[], year text; helper private.is_admin(); trigger fn public.set_updated_at()).
- Produces: final products shape per spec §4; tables product_categories, product_features, product_blog_links; private.is_owner(); role-protection trigger. Later tasks' row types must match these columns exactly.

Step 1: Write the migration file — exact content

```
-- NurvexThink --0003: products experience (spec 2026-07-04 rev 3).
-- Transforms the LIVE schema data-preservingly. Apply in Supabase →SQL Editor
-- after 0001 + 0002. Idempotent: IF NOT EXISTS + DO $$ guards; safe to re-run.

-- =====
-- 1) product_categories (admin-managed; replaces products.category text)
-- =====
create table if not exists public.product_categories (
  id uuid primary key default gen_random_uuid(),
  name text not null unique,
  slug text not null unique,
  sort_order integer not null default 0,
  created_at timestamptz not null default now()
);

alter table public.product_categories enable row level security;
create index if not exists product_categories_slug_idx on public.product_categories (
  slug);

drop policy if exists "categories: public reads" on public.product_categories;
create policy "categories: public reads" on public.product_categories
  for select to anon, authenticated
  using (true);

drop policy if exists "categories: admin writes" on public.product_categories;
create policy "categories: admin writes" on public.product_categories
  for all to authenticated
  using (private.is_admin())
  with check (private.is_admin());

grant select on public.product_categories to anon, authenticated;
grant insert, update, delete on public.product_categories to authenticated;

-- Backfill categories from the existing free-text column, then swap to FK.
do $$
begin
  if exists (select 1 from information_schema.columns
    where table_schema = 'public' and table_name = 'products'
      and column_name = 'category') then
    insert into public.product_categories (name, slug)
    select distinct p.category,
```

```

        trim(both '-' from lower(regex_replace(p.category, '[^a-zA-Z0-9]+', '-', 'g'))
    )
    from public.products p
    where p.category is not null and p.category <> ''
    on conflict (name) do nothing;
end if;
end $$;

alter table public.products
    add column if not exists category_id uuid references public.product_categories (id) on
        delete set null;

do $$
begin
    if exists (select 1 from information_schema.columns
                where table_schema = 'public' and table_name = 'products'
                  and column_name = 'category') then
        update public.products p
        set category_id = c.id
        from public.product_categories c
        where p.category = c.name and p.category_id is null;

        alter table public.products drop column category;
    end if;
end $$;

-- =====
-- 2) status ('Live'/'Beta'/'Soon') → lifecycle ('live'/'beta'/'soon')
-- Guard on the CURRENT check-less 0002 shape: only rename when the values
-- are the 0002 vocabulary (i.e. this hasn't run yet).
-- =====
do $$
begin
    if exists (select 1 from information_schema.columns
                where table_schema = 'public' and table_name = 'products'
                  and column_name = 'status')
        and not exists (select 1 from information_schema.columns
                        where table_schema = 'public' and table_name = 'products'
                          and column_name = 'lifecycle') then
        alter table public.products rename column status to lifecycle;
    end if;
end $$;

update public.products set lifecycle = lower(lifecycle) where lifecycle <> lower(
    lifecycle);
alter table public.products alter column lifecycle set default 'live';
alter table public.products drop constraint if exists products_lifecycle_check;
alter table public.products
    add constraint products_lifecycle_check check (lifecycle in ('live', 'beta', 'soon'));
create index if not exists products_lifecycle_idx on public.products (lifecycle);

-- =====
-- 3) published bool → status ('draft'/'published') + published_at
-- Order matters: backfill BEFORE the CHECK; swap the RLS policy BEFORE
-- dropping the old column it referenced.
-- =====
alter table public.products add column if not exists published_at timestamp;
alter table public.products add column if not exists status text;

do $$

```

```

begin
  if exists (select 1 from information_schema.columns
             where table_schema = 'public' and table_name = 'products'
             and column_name = 'published') then
    update public.products
    set status = case when published then 'published' else 'draft' end
    where status is null;

    update public.products
    set published_at = coalesce(published_at, updated_at)
    where status = 'published';
  end if;
end $$;

update public.products set status = 'draft' where status is null;
alter table public.products alter column status set not null;
alter table public.products alter column status set default 'draft';
alter table public.products drop constraint if exists products_status_check;
alter table public.products
  add constraint products_status_check check (status in ('draft', 'published'));

drop policy if exists "products: public reads published" on public.products;
create policy "products: public reads published" on public.products
  for select to anon, authenticated
  using (status = 'published');

alter table public.products drop column if exists published;
create index if not exists products_status_idx on public.products (status);

-- =====
-- 4) tags → tech
-- =====
do $$
begin
  if exists (select 1 from information_schema.columns
             where table_schema = 'public' and table_name = 'products'
             and column_name = 'tags')
    and not exists (select 1 from information_schema.columns
                   where table_schema = 'public' and table_name = 'products'
                   and column_name = 'tech') then
    alter table public.products rename column tags to tech;
  end if;
end $$;

-- =====
-- 5) New content columns + stable sort_order backfill (10, 20, 30 ...by age)
-- =====
alter table public.products add column if not exists tagline text;
alter table public.products add column if not exists highlights text[] not null default
  '{}';
alter table public.products add column if not exists technical_details text;
alter table public.products add column if not exists gallery text[] not null default
  '{}';
alter table public.products add column if not exists sort_order integer not null default
  0;
alter table public.products add column if not exists seo_description text;
alter table public.products add column if not exists og_image text;

with numbered as (
  select id, row_number() over (order by created_at asc) as rn

```

```

    from public.products
    where sort_order = 0
)
update public.products p
set sort_order = (select coalesce(max(sort_order), 0) from public.products) + n.rn * 10
from numbered n
where p.id = n.id;

create index if not exists products_sort_order_idx on public.products (sort_order);
create index if not exists products_category_id_idx on public.products (category_id);

-- =====
-- 6) product_features (image + explanation blocks, admin-ordered)
-- =====
create table if not exists public.product_features (
    id uuid primary key default gen_random_uuid(),
    product_id uuid not null references public.products (id) on delete cascade,
    title text not null,
    description text,
    image text,
    sort_order integer not null default 0,
    created_at timestamptz not null default now()
);

alter table public.product_features enable row level security;
create index if not exists product_features_product_idx
    on public.product_features (product_id, sort_order);

drop policy if exists "features: public reads for published products" on public.
    product_features;
create policy "features: public reads for published products" on public.product_features
    for select to anon, authenticated
    using (exists (
        select 1 from public.products p
        where p.id = product_id and p.status = 'published'
    ));

drop policy if exists "features: admin reads all" on public.product_features;
create policy "features: admin reads all" on public.product_features
    for select to authenticated
    using (private.is_admin());

drop policy if exists "features: admin writes" on public.product_features;
create policy "features: admin writes" on public.product_features
    for all to authenticated
    using (private.is_admin())
    with check (private.is_admin());

grant select on public.product_features to anon, authenticated;
grant insert, update, delete on public.product_features to authenticated;

-- =====
-- 7) product_blog_links (ordered many-to-many; both-sides-published for anon)
-- =====
create table if not exists public.product_blog_links (
    product_id uuid not null references public.products (id) on delete cascade,
    blog_post_id uuid not null references public.blog_posts (id) on delete cascade,
    sort_order integer not null default 0,
    primary key (product_id, blog_post_id)
);

```

```

alter table public.product_blog_links enable row level security;
create index if not exists product_blog_links_product_idx
  on public.product_blog_links (product_id);
create index if not exists product_blog_links_post_idx
  on public.product_blog_links (blog_post_id);

drop policy if exists "links: public reads published pairs" on public.product_blog_links
  ;
create policy "links: public reads published pairs" on public.product_blog_links
  for select to anon, authenticated
  using (
    exists (select 1 from public.products p
              where p.id = product_id and p.status = 'published')
    and exists (select 1 from public.blog_posts b
                 where b.id = blog_post_id and b.status = 'published')
  );

drop policy if exists "links: admin reads all" on public.product_blog_links;
create policy "links: admin reads all" on public.product_blog_links
  for select to authenticated
  using (private.is_admin());

drop policy if exists "links: admin writes" on public.product_blog_links;
create policy "links: admin writes" on public.product_blog_links
  for all to authenticated
  using (private.is_admin())
  with check (private.is_admin());

grant select on public.product_blog_links to anon, authenticated;
grant insert, update, delete on public.product_blog_links to authenticated;

-- =====
-- 8) Security fix: only owners may change profiles.role (spec §5).
-- A trigger, not RLS --RLS cannot see WHICH column changed. Contexts with
-- no auth.uid() (SQL editor, service_role maintenance) stay allowed.
-- =====
create or replace function private.is_owner()
returns boolean
language sql
security definer
set search_path = ''
stable
as $$
  select exists (
    select 1
    from public.profiles p
    where p.id = (select auth.uid())
      and p.role = 'owner'
  );
$$;

revoke all on function private.is_owner() from public, anon;
grant execute on function private.is_owner() to authenticated;

create or replace function public.protect_profile_role()
returns trigger
language plpgsql
security definer
set search_path = ''

```

```

as $$
begin
  if new.role is distinct from old.role
    and (select auth.uid()) is not null
    and not private.is_owner() then
    raise exception 'only owners can change roles';
  end if;
  return new;
end;
$$;

drop trigger if exists protect_profile_role on public.profiles;
create trigger protect_profile_role
  before update on public.profiles
  for each row execute function public.protect_profile_role();

```

Step 2: Sanity-check the SQL statically

Run:

```

node -e "const s=require('fs').readFileSync('supabase/migrations/0003_products_experience.sql','utf8'); const b=(s.match(/do \$\$/g)||[]).length, e=(s.match(/end \$\$/g)||[]).length; console.log('D0 blocks', b, '== END blocks', e, b===e ? 'OK' : 'MISMATCH');"

```

Expected: D0 blocks 5 == END blocks 5 OK

Step 3: Commit

```

git add supabase/migrations/0003_products_experience.sql
git commit -m "feat: migration 0003 --categories, lifecycle/status split, features, blog links, role guard"

```

3 Task 2: TypeScript Layer — Row Types, Public Queries (TDD), Admin Surfaces

One compile unit: the row-type rename breaks every consumer, so all files land in this task; the gate at the end is the full green bar.

Files

- Modify: `src/lib/supabase/types.ts` (whole file replace)
- Modify: `src/lib/queries.ts`:1-65
- Create: `src/lib/queries.test.ts`
- Modify: `src/lib/admin-queries.ts`:2,7,16
- Modify: `src/app/admin/(panel)/products/actions.ts`:9-31,74-80
- Modify: `src/components/admin/product-form.tsx`:6,29,60-77,94-108,110-129
- Modify: `src/app/admin/(panel)/products/page.tsx`:52-68

Interfaces

- **Consumes:** column names produced by Task 1's migration (exact: lifecycle, status, tech, tagline, highlights, technical_details, gallery, sort_order, seo_description, og_image, published_at, category_id).
- **Produces:** ProductRow, ProductCategoryRow, ProductFeatureRow, ProductBlogLinkRow types; exported toProduct(row: ProductRow): Product. PR 2/3 build on these names.

Step 1: Write the failing test — create src/lib/queries.test.ts

```
import { describe, it, expect } from "vitest";
import { toProduct } from "@lib/queries";
import type { ProductRow } from "@lib/supabase/types";

const row: ProductRow = {
  id: "00000000-0000-0000-0000-000000000001",
  slug: "fluxboard",
  name: "FluxBoard",
  tagline: null,
  summary: "A keyboard-first project board.",
  highlights: [],
  description: "Long text",
  technical_details: null,
  cover_image: null,
  gallery: [],
  category_id: null,
  category_name: "Productivity",
  tech: ["Next.js", "Realtime"],
  lifecycle: "beta",
  year: "2026",
  live_url: "#",
  repo_url: null,
  featured: true,
  sort_order: 10,
  status: "published",
  seo_description: null,
  og_image: null,
  published_at: "2026-07-01T00:00:00Z",
  created_at: "2026-06-01T00:00:00Z",
  updated_at: "2026-07-01T00:00:00Z",
};

describe("toProduct", () => {
  it("maps lifecycle to the UI status label", () => {
    expect(toProduct(row).status).toBe("Beta");
    expect(toProduct({ ...row, lifecycle: "live" }).status).toBe("Live");
    expect(toProduct({ ...row, lifecycle: "soon" }).status).toBe("Soon");
  });

  it("maps tech to the UI tags array", () => {
    expect(toProduct(row).tags).toEqual(["Next.js", "Realtime"]);
  });

  it("keeps the existing null-safety defaults", () => {
    const bare = { ...row, summary: null, description: null, year: null, live_url: null,
      category_name: null };
    const p = toProduct(bare);
    expect(p.summary).toBe("");
  });
});
```

```

    expect(p.description).toBe("");
    expect(p.year).toBe("");
    expect(p.liveUrl).toBe("#");
    expect(p.category).toBe("Software");
  });
});

```

Step 2: Run test to verify it fails

Run: `npx vitest run src/lib/queries.test.ts`

Expected: FAIL — `toProduct` is not exported / `ProductRow` does not exist.

Step 3: Replace `src/lib/supabase/types.ts` entirely with

```

/**
 * Database types for the NurvexThink Supabase schema.
 * Mirrors supabase/migrations/0001-0003. If you change the schema,
 * update this file (or regenerate with 'supabase gen types typescript').
 */

export type ProductRow = {
  id: string;
  slug: string;
  name: string;
  tagline: string | null;
  summary: string | null;
  highlights: string[];
  description: string | null;
  technical_details: string | null;
  cover_image: string | null;
  gallery: string[];
  category_id: string | null;
  /** Denormalized on select via 'product_categories(name)' join; not a column. */
  category_name?: string | null;
  tech: string[];
  lifecycle: "live" | "beta" | "soon";
  year: string | null;
  live_url: string | null;
  repo_url: string | null;
  featured: boolean;
  sort_order: number;
  status: "draft" | "published";
  seo_description: string | null;
  og_image: string | null;
  published_at: string | null;
  created_at: string;
  updated_at: string;
};

export type ProductCategoryRow = {
  id: string;
  name: string;
  slug: string;
  sort_order: number;
  created_at: string;
};

export type ProductFeatureRow = {

```

```
id: string;
product_id: string;
title: string;
description: string | null;
image: string | null;
sort_order: number;
created_at: string;
};

export type ProductBlogLinkRow = {
  product_id: string;
  blog_post_id: string;
  sort_order: number;
};

export type BlogPostRow = {
  id: string;
  slug: string;
  title: string;
  category: string | null;
  excerpt: string | null;
  content: string | null;
  cover_image: string | null;
  author_id: string | null;
  author_name: string | null;
  reading_time: string | null;
  status: "draft" | "published";
  published_at: string | null;
  created_at: string;
  updated_at: string;
};

export type OrderRow = {
  id: string;
  name: string;
  email: string;
  company: string | null;
  project_type: string | null;
  budget: string | null;
  details: string;
  status: "new" | "contacted" | "closed";
  created_at: string;
};

export type ProfileRow = {
  id: string;
  email: string | null;
  full_name: string | null;
  role: "owner" | "admin";
  created_at: string;
};

export type Database = {
  public: {
    Tables: {
      profiles: {
        Row: ProfileRow;
        Insert: Partial<ProfileRow> & { id: string };
        Update: Partial<ProfileRow>;
        Relationships: [];
```

```

};
products: {
  Row: ProductRow;
  Insert: { slug: string; name: string } & Partial<
    Omit<ProductRow, "slug" | "name" | "category_name">
  >;
  Update: Partial<Omit<ProductRow, "category_name">>;
  Relationships: [];
};
product_categories: {
  Row: ProductCategoryRow;
  Insert: { name: string; slug: string } & Partial<
    Omit<ProductCategoryRow, "name" | "slug">
  >;
  Update: Partial<ProductCategoryRow>;
  Relationships: [];
};
product_features: {
  Row: ProductFeatureRow;
  Insert: { product_id: string; title: string } & Partial<
    Omit<ProductFeatureRow, "product_id" | "title">
  >;
  Update: Partial<ProductFeatureRow>;
  Relationships: [];
};
product_blog_links: {
  Row: ProductBlogLinkRow;
  Insert: ProductBlogLinkRow;
  Update: Partial<ProductBlogLinkRow>;
  Relationships: [];
};
blog_posts: {
  Row: BlogPostRow;
  Insert: { slug: string; title: string } & Partial<Omit<BlogPostRow, "slug" | "
    title">>;
  Update: Partial<BlogPostRow>;
  Relationships: [];
};
orders: {
  Row: OrderRow;
  Insert: {
    name: string;
    email: string;
    details: string;
    company?: string | null;
    project_type?: string | null;
    budget?: string | null;
  };
  Update: Partial<OrderRow>;
  Relationships: [];
};
};
Views: Record<string, never>;
Functions: Record<string, never>;
Enums: Record<string, never>;
};
};

```

Step 4: Update `src/lib/queries.ts`

Replace lines 1–65 (imports, `toProduct`, `getProducts`, `getProductBySlug`); `toPost/getPosts/getPostBySlug` stay as they are:

```
import { createPublicSupabaseClient } from "@lib/supabase/public";
import type { Product, BlogPost } from "@lib/content";
import type { ProductRow, BlogPostRow } from "@lib/supabase/types";

const LIFECYCLE_LABEL = { live: "Live", beta: "Beta", soon: "Soon" } as const;

/** Select that denormalizes the category name onto the row. */
const PRODUCT_SELECT = `*, product_categories(name)`;

type ProductRowJoined = ProductRow & {
  product_categories?: { name: string } | null;
};

export function toProduct(row: ProductRowJoined): Product {
  return {
    slug: row.slug,
    name: row.name,
    category: row.product_categories?.name ?? row.category_name ?? "Software",
    summary: row.summary ?? "",
    description: row.description ?? "",
    status: LIFECYCLE_LABEL[row.lifecycle] ?? "Live",
    tags: row.tech ?? [],
    year: row.year ?? "",
    liveUrl: row.live_url ?? "#",
  };
}

export async function getProducts(): Promise<Product[]> {
  try {
    const supabase = createPublicSupabaseClient();
    const { data, error } = await supabase
      .from("products")
      .select(PRODUCT_SELECT)
      .eq("status", "published")
      .order("featured", { ascending: false })
      .order("sort_order", { ascending: true })
      .order("created_at", { ascending: false });
    if (error || !data) return [];
    return data.map(toProduct);
  } catch {
    return [];
  }
}

export async function getProductBySlug(slug: string): Promise<Product | null> {
  try {
    const supabase = createPublicSupabaseClient();
    const { data, error } = await supabase
      .from("products")
      .select(PRODUCT_SELECT)
      .eq("slug", slug)
      .eq("status", "published")
      .maybeSingle();
    if (error || !data) return null;
    return toProduct(data);
  }
}
```

```

    } catch {
      return null;
    }
  }
}

```

Step 5: Run the mapping test — verify it passes

Run: `npx vitest run src/lib/queries.test.ts`

Expected: PASS (3 tests)

Step 6: Update `src/lib/admin-queries.ts` — only the type name changes (3 spots)

```

Line 2: import type { ProductRow, BlogPostRow, OrderRow } from "@lib/supabase/types";
Line 7: export async function listProductsAdmin(): Promise<ProductRow[]> {
Line 16: export async function getProductAdmin(id: string): Promise<ProductRow | null> {

```

Step 7: Update `src/app/admin/(panel)/products/actions.ts`

Replace `parseForm` (lines 9–31) and `toggleProductPublished` (lines 74–80):

```

function parseForm(formData: FormData) {
  const tech = String(formData.get("tech") ?? "").split(",")
    .map((t) => t.trim())
    .filter(Boolean);
  const lifecycleRaw = String(formData.get("lifecycle") ?? "live").toLowerCase();
  const published = String(formData.get("published") === "on");
  const existingPublishedAt = String(formData.get("existing_published_at") ?? "").trim() || null;
  return {
    slug: String(formData.get("slug") ?? "").trim().toLowerCase(),
    name: String(formData.get("name") ?? "").trim(),
    summary: String(formData.get("summary") ?? "").trim() || null,
    description: String(formData.get("description") ?? "").trim() || null,
    live_url: String(formData.get("live_url") ?? "").trim() || null,
    year: String(formData.get("year") ?? "").trim() || null,
    lifecycle: (["live", "beta", "soon"].includes(lifecycleRaw) ? lifecycleRaw : "live")
      as "live" | "beta" | "soon",
    tech,
    featured: String(formData.get("featured") === "on"),
    status: (published ? "published" : "draft") as "draft" | "published",
    published_at: published ? (existingPublishedAt ?? new Date().toISOString()) : existingPublishedAt,
  };
}

```

```

export async function toggleProductPublished(formData: FormData) {
  const id = String(formData.get("id") ?? "");
  const next = String(formData.get("current") !== "true");
  const supabase = await createServerSupabaseClient();
  const { data: row } = await supabase
    .from("products")
    .select("published_at")
    .eq("id", id)

```

```

    .maybeSingle();
    await supabase
      .from("products")
      .update({
        status: next ? "published" : "draft",
        published_at: next ? (row?.published_at ?? new Date().toISOString()) : row?.
          published_at,
      })
      .eq("id", id);
    revalidateAll();
  }

```

Note: the free-text category field is dropped from the form in this PR — the category becomes a managed table; the full category picker arrives with the tiered editor in PR 3. createProduct/updateProduct bodies are unchanged — parseForm's return feeds them.

Step 8: Update `src/components/admin/product-form.tsx`

```

Line 6: import type { ProductRow } from "@lib/supabase/types";
Line 29: product?: ProductRow;

```

Replace the Category/Status/Year grid (lines 60–77) with (category input removed):

```

<div className="grid gap-5 sm:grid-cols-2">
  <label className={labelClass}>
    Lifecycle
    <select
      name="lifecycle"
      defaultValue={product?.lifecycle ?? "live"}
      className={fieldClass}
    >
      <option value="live">Live</option>
      <option value="beta">Beta</option>
      <option value="soon">Soon</option>
    </select>
  </label>
  <label className={labelClass}>
    Year
    <input name="year" defaultValue={product?.year ?? ""} className={fieldClass} />
  </label>
</div>

```

In the Tags/Live URL grid (lines 94–108), rename the field to tech:

```

<label className={labelClass}>
  Tech <span className="text-muted-foreground font-normal">(comma separated)</span>
  <input
    name="tech"
    defaultValue={(product?.tech ?? []).join(", ")}
    placeholder="Next.js, Realtime"
    className={fieldClass}
  />
</label>

```

Replace the Published checkbox (lines 120–128) and add the hidden `existing_published_at` just before it:

```

<input
  type="hidden"
  name="existing_published_at"

```

```

      value={product?.published_at ?? ""}
    />
    <label className="flex items-center gap-2 text-sm font-medium">
      <input
        type="checkbox"
        name="published"
        defaultChecked={product?.status === "published"}
        className="size-4"
      />
      Published (visible on the site)
    </label>

```

Step 9: Update `src/app/admin/(panel)/products/page.tsx` (lines 52–68)

```

Line 52 (lifecycle cell): <td className="text-muted-foreground px-4 py-3 capitalize">{p.
  lifecycle}</td>
Line 56 (hidden current): <input type="hidden" name="current" value={String(p.status ===
  "published")} />

```

Lines 61 & 66 (badge): replace `p.published` with `p.status === "published"` in both the `cn(...)` condition and the label (`{p.status === "published" ? "Published" : "Hidden"}`). Also line 49 shows `p.category` — the column is gone; replace that fragment with nothing: `{p.slug}` only (delete `{p.category ? `·${p.category}` : ""}`).

Step 10: Full green bar

Run: `npm run typecheck && npm run lint && npm test && npm run build`

Expected: all exit 0; test count grows by 3 (the new `queries.test.ts`).

Step 11: Commit

```

git add src/lib/supabase/types.ts src/lib/queries.ts src/lib/queries.test.ts src/lib/
  admin-queries.ts "src/app/admin/(panel)/products/actions.ts" src/components/admin/
  product-form.tsx "src/app/admin/(panel)/products/page.tsx"
git commit -m "feat: point queries and admin CRUD at the 0003 schema (lifecycle/status/
  tech)"

```

4 Task 3: Apply Migration, Verify Live, Document, PR

Files

- Modify: `CLAUDE.md` (Data model line)

Interfaces

- Consumes: Task 1's SQL file; Task 2's green build.
- Produces: live DB on the new schema; open PR for `feat/products`.

Step 1: Update `CLAUDE.md` data model line

Replace:

```
Tables: 'profiles', 'products', 'blog_posts', 'orders'. Storage: 'product-images',
```

with:

Tables: 'profiles', 'products', 'product_categories', 'product_features', 'product_blog_links', 'blog_posts', 'orders'. Storage: 'product-images',

Step 2: HUMAN GATE — apply 0003 to the live DB

Paste supabase/migrations/0003_products_experience.sql into <https://supabase.com/dashboard/project/axbsghyqhhdaiylcksbv> → SQL Editor → Run. (Or hand it to Claude once the Supabase MCP server is authenticated.)

Timing note: production still runs the old code until this PR deploys; between applying 0003 and the deploy finishing, the live /products grid renders its fail-soft empty state (queries catch and return []). Pre-launch this is acceptable — apply the migration, then merge immediately.

Step 3: Verify the migration

Run in the SQL editor:

```
select
  (select count(*) from public.products) as products,
  (select count(*) from public.products where status = 'published') as published,
  (select count(*) from public.product_categories) as categories,
  (select count(*) from public.products where category_id is null) as uncategorized,
  (select count(*) from public.products where sort_order = 0) as unsorted;
```

Expected: products = 6, published = 6, categories = 6 (Productivity, Analytics, Finance, AI, Developer tools, Security), uncategorized = 0, unsorted = 0.

Also confirm the columns:

```
select column_name from information_schema.columns
where table_schema = 'public' and table_name = 'products' order by column_name;
```

Expected to include lifecycle, status, tech, tagline, highlights, technical_details, gallery, sort_order, seo_description, og_image, published_at — and NOT published, tags, category.

The live DB has no owner yet (0001 defaults every profile to admin) — promote the CEO while in the SQL editor:

```
update public.profiles set role = 'owner' where email = 'fatima.abdulraheemdev.17@gmail.com';
select email, role from public.profiles;
```

Step 4: Push branch + open PR

```
git push -u origin feat/products
gh pr create --title "feat: products schema --categories, lifecycle/status, features, blog links" --body "PR 1 of 3 from docs/superpowers/specs/2026-07-04-products-experience-design.md (§4, §8.1).

- Migration 0003: data-preserving transform of the live schema (6 seed products survive)
- product_categories / product_features / product_blog_links tables + RLS + indexes
- profiles.role privilege-escalation fix (owner-gated trigger)
- TS layer repointed (lifecycle/status/tech); public UI renders identically; basic admin CRUD keeps working

[bot]Generated with [Claude Code](https://claude.com/claude-code)"
```

Step 5: Post-merge smoke check

After merge + deploy, `/products` on production shows the same six cards as before (order may differ only if `featured/created_at` ties resolved differently — it should not), and `/admin/products` still lists, edits, and toggles visibility.

5 Self-Review (done)

- **Spec coverage (PR 1 scope):** §4 migration path steps 1–7 → Task 1 sections 1–8; §5 trigger fix → Task 1 §8; “queries/view-models keep rendering unchanged” → Task 2 (UI `Product` shape untouched, verified by unchanged public components + green bar); indexes → Task 1 (all listed). Quick View/features UI, sitemap, tiered editor = PR 2/3 (own plans).
- **Placeholder scan:** none — every step carries full code/commands.
- **Type consistency:** `ProductRow.lifecycle`: `"live"|"beta"|"soon"` ↔ migration CHECK ↔ `LIFECYCLE_LABEL` keys ↔ form option values; `status`: `"draft"|"published"` ↔ CHECK ↔ actions/page comparisons; `tech` ↔ form field `name="tech"` ↔ `parseForm` reads `tech`. `category_name` is optional on `ProductRow` (join alias), excluded from `Insert/Update`.